KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Parzi and divines eye: local knowledge base and geospatial intelligence

Project ID: PRJ-2026-059

Team ID: T059

Team GitHub Repository: https://github.com/hharasudhan122-ai/Gen-ai-project-

Student Name	Registration Number	Individual GitHub Link
Hari Hara Sudhan M	99230041113	https://github.com/hharasudhan122-ai/gen-ai-project-and-assignement
Rebinesh	99230041147	https://github.com/Rebinesh18/ICL-GEN-AI
Janisha A	99230041145	https://github.com/Janisha41145/Gen-AI
Manoj Kumar N	99230041120	https://github.com/manoj14666/Gen-ai-projects-and-assignments.git
Annalakshmi M	99230041119	https://github.com/annubommi06-crypto/gen-ai-project-and-assignment

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

Section	Topic
1	Introduction
2	Problem Statement and Objectives
3	System Architecture and RAG Workflow
4	Implementation Details and Technology Stack
5	Tools Integration, Testing and Performance
6	Industry Readiness, Project Outcomes and Future Scope

1. Introduction

Parzi: Divine's Eye is a multimodal AI assistant designed to bring conversation, document search, desktop automation and 3D geospatial information into one workspace. Its interface is built around a browser-based command dashboard, with a responsive visual assistant called the Jarvis Orb and a 3D Earth environment referred to as Divine's Eye.

The project follows a privacy-first approach. The browser application, known as Parzi Core, handles the user interface and the local document-retrieval workflow, while a Node.js companion process provides a bridge to selected desktop and Android functions. The system is intended to work locally wherever possible, with public data feeds requiring an internet connection when live information is requested.

The RAG feature is one part of the wider platform. In the current prototype description, PDF and text documents are parsed in the browser, converted into dense embeddings with @xenova/transformers, and searched using cosine similarity against an in-memory vector index. The retrieved passages can then be used as context for an assistant response. The project description does not specify a particular language model or a persistent vector database, so neither is assumed in this document.

Alongside document search, Divine's Eye brings together public information about aircraft, ships, satellites and seismic activity in a shared 3D view. The aim is to reduce the need to move between separate tools and give operators a single place to explore information through both voice and visual interaction.

2. Problem Statement and Objectives

OSINT and automation tasks often involve several unrelated websites, command-line tools and desktop applications. Moving between them takes time and makes it harder to maintain a clear picture of what is happening. The problem becomes more significant when information comes from different sources, updates at different intervals or needs to be compared spatially.

There is also a privacy concern when documents are uploaded to external AI services for processing. For sensitive workspaces, keeping document parsing and embedding generation on the local device can reduce the need to send document contents to remote services. Traditional voice assistants also offer limited control over spatial interfaces and local desktop or mobile workflows.

The main objectives of Parzi: Divine's Eye are to:

- Bring conversational interaction and multiple public geospatial data layers into one command dashboard.
- Allow users to load PDF and text documents and retrieve relevant passages through local semantic search.

- Generate document embeddings in the browser using WebGPU or WebAssembly where supported.
- Provide voice interaction with audio filtering and visual feedback through the Jarvis Orb.
- Connect the browser interface to a local companion process for supported desktop and Android actions.
- Display aircraft, maritime, satellite and seismic information in a 3D Earth environment.
- Retain useful workspace information locally and clearly identify simulated or reconstructed telemetry.

3. System Architecture and RAG Workflow

3.1 Overall application architecture

Parzi uses a two-part architecture. The browser frontend, Parzi Core, is responsible for the HUD, voice interaction, document handling and visual presentation. A separate Node.js process, companion-server.js, runs locally and provides the connection between the browser and selected native system functions. The two parts communicate through REST and WebSocket protocols on 127.0.0.1:7890.

Part	Responsibility
Parzi Core	Browser interface, Jarvis Orb, voice interaction, document parsing, local embeddings and retrieval, browser storage and the 3D visualization interface.
Companion engine	Local Node.js service that handles the browser connection and supported desktop/mobile automation requests.
Public data sources	Aircraft, maritime, satellite, seismic and public camera information. Live availability depends on the relevant source.
Device bridge	ADB over USB or Wi-Fi for supported Android actions, together with operating-system-specific desktop application launching.

3.2 Document ingestion and retrieval

The RAG workflow begins when a user selects a document. The prototype described for this project supports PDF and text-file parsing within the browser environment. The extracted text is then prepared for embedding and added to an in-memory vector index.

- Document loading: PDF and text files are read in the browser. PDF.js is included among the locally vendored libraries; Mammoth.js is also included in the project runtime for document handling.
- Text preparation: The extracted text is organized for embedding. A fixed chunk size, overlap value and metadata format have not been specified in the project details.
- Embedding generation: @xenova/transformers is used to generate dense vector embeddings on the client, with WebGPU and WebAssembly runtimes described as execution options.
- Local indexing: The prototype keeps the vectors in an in-memory index and uses cosine similarity to compare them.
- Query retrieval: A user query is embedded in the same vector space and compared with the stored document vectors to find relevant passages.
- Answer context: Retrieved passages can be passed to the assistant as supporting context. The exact language model, prompt format, top-K setting and citation method are not specified in the supplied project description.

3.3 Scope of the current RAG prototype

The implemented RAG work described here covers document parsing, client-side embedding generation and cosine-similarity retrieval. The project information does not identify a named vector database, a persistent vector store or a specific generative model. It also does not provide a fixed chunking configuration or measured retrieval-accuracy results. These details should be added only when the final implementation has been confirmed.

3.4 Geospatial data flow

The geospatial side of the platform combines several types of public telemetry. Aircraft information is obtained from ADS-B sources such as OpenSky or ADSBx, while maritime information uses AIS data. USGS feeds provide seismic information, and satellite paths are calculated from NORAD Two-Line Element sets using SGP4 propagation. Public CCTV sources are also included in the described data pipeline.

Incoming data is normalized into a common spatial coordinate system for display on the 3D globe. A polling and interpolation buffer smooths movement between updates. When a live feed is unavailable, the display is designed to identify reconstructed or simulated information with labels such as RECONSTRUCTED ESTIMATE or SIMULATED, rather than presenting it as a live observation.

The globe uses WebGL-based rendering and high-precision terrain visualization, with atmospheric lighting and sun/shadow effects. Operators can follow selected aircraft or orbital assets, view telemetry overlays and add spatial markers through the voice-to-geometry workflow described for the project.

4. Implementation Details and Technology Stack

The project uses standard web technologies for the browser interface and a local Node.js process for system integration. The main tools and their stated roles are summarized below.

Area	Technology	Use in the project
Frontend	HTML5, CSS3, Vanilla ES6+ JavaScript	Browser-based HUD, responsive layouts and client-side interaction.
Local service	Node.js, companion-server.js, REST and WebSocket	Communication between the browser and native desktop/mobile functions.
Build and serving	Vite; Python local microserver environments	Development, build and local serving workflow.
RAG and embeddings	@xenova/transformers, WebGPU, WebAssembly, cosine similarity	Browser-side dense embeddings and in-memory semantic retrieval.
Document libraries	PDF.js, Mammoth.js, Marked, DOMPurify, KaTeX	Document handling, Markdown parsing, sanitization and mathematical notation.
3D and geospatial	WebGL, CesiumJS, Three.js	Globe and 3D scene rendering. The project description names both engines without assigning every feature to one specific engine.
Orbital tracking	SGP4 and NORAD TLE data	Satellite trajectory propagation.
Audio and speech	Web Audio API, Web Speech API, TTS queues	Audio filtering, speech interaction and spoken responses.
Local storage	IndexedDB and HTML5 LocalStorage	Conversation persistence, local contact graphs and workspace settings.
Android integration	ADB over USB/Wi-Fi	Supported Android application dispatch and device interaction.

Area	Technology	Use in the project
Desktop integration	PowerShell/CLI, macOS open, Linux xdg-open and URI protocols	Launching registered desktop applications and supported system targets.
Graphs and diagrams	D3.js and Mermaid.js	Relational graphs and technical/topological diagrams.

4.1 HUD and voice interaction

The interface uses a dark glassmorphic design, with CSS transforms, backdrop filters and responsive layouts. A starfield canvas provides background movement. At the centre of the voice interface is the Jarvis Orb, drawn through a 2D Canvas rendering pipeline. It uses concentric sine-wave rings and a particle field, with visual changes for the Idle, Listening, Thinking and Speaking states.

Voice input is connected to the Web Speech API, while spoken responses are handled through a text-to-speech audio queue. Before voice activity detection, the Web Audio API applies a 300 Hz–3400 Hz bandpass filter. This is intended to reduce unwanted low- and high-frequency sound, including some common background noise. The supplied description does not detail a complete speaker-identification or biometric verification algorithm.

4.2 Document handling and security

Runtime libraries are stored locally rather than loaded from a CDN. The named libraries include KaTeX, Marked, DOMPurify, PDF.js and Mammoth.js. DOMPurify is used to sanitize content before it is rendered, helping reduce the risk of script injection through document-derived or formatted text. The project description reports testing the rendering pipeline against OWASP XSS attack vectors, although detailed test cases and sanitizer policies were not provided.

4.3 Desktop and mobile automation

The companion process supports application launching through the mechanisms available on Windows, macOS and Linux. The project uses an application mapping called PC_APP_MAP to associate supported desktop applications and URI schemes with launch actions. For Android, the ADB bridge communicates over USB or Wi-Fi and uses a PHONE_APP_MAP to identify supported applications.

The mobile touch automation layer uses normalized screen coordinates, represented by xPct and yPct, so that taps can be positioned relative to the device display. The project description includes WhatsApp communication and calling workflows among its demonstrations. Public telemetry monitoring and local command execution are kept as separate operating modes to reduce the chance of an unintended system action.

4.4 3D assets and visual overlays

The 3D asset hangar includes glTF/GLB models identified in the project brief: Boeing 787, ATR-72, Citation, Bell 206 and MQ-9 Reaper. These models replace simpler radar symbols as the camera zooms in. The visualization also includes custom GLSL effects for FLIR, night vision, noir, CRT scanlines and a high-altitude snow view. Telemetry overlays can show target boxes, identification details, velocity vectors, altitude and bearing.

5. Tools Integration, Testing and Performance

5.1 Integration of telemetry and visualization

The platform brings aircraft, maritime, satellite and seismic feeds into the same geospatial view. Each source provides information in its own format and update cycle, so the data pipeline normalizes the incoming coordinates before displaying them together. The interpolation buffer helps keep moving assets visually smooth between polling updates.

The camera can follow selected aircraft or orbital objects, with coordinate interpolation and terrain-following altitude clamping used for the ride-along view. Operators can also use the voice-to-geometry feature to describe polygon coordinates, boundaries and markers for placement on the 3D globe.

5.2 Testing and performance work described

The project brief records the following testing and optimization activities. The figures below are reported project results; detailed test logs and hardware configurations were not included in the supplied material.

Area	Work or result reported
Voice and background noise	Voice recognition was tested under simulated 45 dB–65 dB background noise. The brief reports fewer false triggers after narrow-band filtering; it does not provide a numerical hit-rate table.
Rendering performance	Chrome DevTools heap profiling, GPU draw-call batching and geometry LOD pruning were used. A steady 60 FPS on standard desktop GPUs is reported, but exact hardware and scene settings are not listed.
Local latency	Desktop application dispatch is reported at under 50 ms, and local vector search at under 200 ms for 100-page documents. The measurement method and test hardware are not specified.
Document security	PDF/Markdown rendering was tested against OWASP XSS attack vectors, with DOMPurify used for sanitization. No formal security audit report is identified.
Android and desktop compatibility	The ADB bridge is reported as tested on Android 10 through 14. Desktop launch scripts were checked on Windows 10/11, macOS Darwin and Ubuntu Linux.
Browser support	WebGPU, WebAssembly and Web Audio API behavior was checked on Chrome, Edge, Brave and Firefox. Exact browser versions and feature-by-feature results are not listed.
Telemetry fallback	When live transponder feeds disconnect, the system is designed to show reconstructed or simulated layers with explicit labels.

5.3 Demonstrations

The project description records demonstrations involving commercial transatlantic air corridors, maritime vessel tracking and voice-triggered WhatsApp dispatch workflows. These examples show how the geospatial dashboard and local automation bridge are intended to work together. Demonstration dates, recordings and formal acceptance criteria were not included in the supplied details.

5.4 Further evidence for evaluation

For a complete technical evaluation, the project can be supported with benchmark logs, test-device specifications, browser and operating-system versions, telemetry timestamps, audio test conditions and demonstration recordings. A separate RAG evaluation would also be useful, covering retrieval relevance, answer grounding and response time on a documented document set. No RAG accuracy percentage or hallucination-reduction figure is stated here because none was provided in the project description.

6. Industry Readiness, Project Outcomes and Future Scope

6.1 Potential areas of use

Parzi is designed to run on a range of hardware, from laptops with integrated graphics to multi-screen command workstations. Its local-first document workflow and modular interface make it relevant to environments where privacy and control over data handling are important. The wider platform may support OSINT research, aerial and maritime monitoring, disaster awareness, emergency response and logistics operations.

USGS seismic feeds and FIRMS wildfire satellite data are identified as useful sources for emergency-response scenarios. The project also describes a JSON adapter approach through which additional radar, IoT or logistics telemetry sources could be connected. The availability of live public data depends on network access; an offline or air-gapped installation would need its required maps, models, terrain assets and data files available locally.

6.2 Privacy and deployment considerations

Keeping document parsing and embedding generation in the browser reduces the need to upload source documents to a remote embedding service. However, the privacy of a full deployment also depends on other components, including map services, telemetry providers, optional model endpoints, logs, software updates and device connections. The project is designed with GDPR- and HIPAA-conscious use cases in mind, but no formal compliance certification or legal assessment was included in the supplied project information.

Before wider deployment, the project documentation should clearly describe local service permissions, ADB pairing, command allowlists, audit logs, update procedures and which features require an internet connection. These points are important deployment considerations rather than claims about controls not described in the current project brief.

6.3 Project outcomes

The project is described as a functioning local prototype that combines a conversational HUD with a multi-layer 3D geospatial engine. Its RAG component demonstrates browser-side document parsing, local embedding generation and semantic retrieval using cosine similarity. The wider platform adds voice interaction, public telemetry visualization, orbital tracking and local desktop/mobile automation.

Two research paper topics have been proposed: “Single-Pane Spatial Awareness: Integrating Multimodal Conversational AI with Real-Time WebGL Geospatial OSINT” and “Client-Side In-Browser Vector Embeddings and RAG Architecture for Air-Gapped Intelligence Analysis.” These are paper candidates, not publications.

The project also identifies two possible patent directions: voice-based control of 3D camera frames and tactical overlays, and a hybrid automation method linking browser AI prompts with local ADB screen coordinates. These are potential innovation areas; patentability has not been established.

6.4 Future scope

- Extend the interface toward WebXR headsets, including Apple Vision Pro and Meta Quest.
- Develop background multi-agent monitors for satellite tracking.
- Expand document retrieval to support images and charts within PDFs.
- Develop the JSON sensor-adapter ecosystem for additional radar, IoT and telemetry sources.
- Prepare Electron or native desktop packaging for simpler installation and deployment.
- Add repeatable RAG evaluation with retrieval-quality measures, answer-grounding checks and documented latency tests.

6.5 Conclusion

Parzi: Divine’s Eye brings several workflows that are usually handled separately into one local-first workspace. The RAG component allows users to parse PDF and text documents in the browser, generate embeddings locally and retrieve relevant passages through cosine similarity. Around this, the platform adds a voice-driven HUD, a 3D geospatial view, public telemetry layers and a local bridge for supported desktop and Android actions.

The prototype establishes a broad foundation for further development. The next step is to document the implementation in greater detail and support the reported performance with repeatable tests. In particular, publishing the exact RAG configuration, evaluation dataset and retrieval result s would make it easier for others to assess and reproduce the document-assistant component.